

TLCTC Framework

Version 2.1 Full Extension Specification

Transit Boundary Operator and Intra-System Boundary Operator

Complete normative draft

Prepared as a full assistant-authored specification for review and adaptation

Status	Assistant Draft — full normative profile
Compatibility	Backward-compatible with TLCTC v2.0
Intended use	Specification review, pilot implementation, parser/profile design, forensic notation
Design stance	Minimal semantic expansion; strong anti-drift guardrails
Date	March 2026

This document specifies the v2.1 notation extensions that I would standardize for TLCTC. It is intentionally stricter than an exploratory draft. The goal is to add structural precision where modern attack paths cross third-party transit infrastructure and internal system containment boundaries, while preserving TLCTC’s cause-oriented model and its ten-cluster classification discipline.

Contents

1. 1. Abstract
2. 2. Status, scope, and relationship to TLCTC v2.0
3. 3. Normative language and conformance philosophy
4. 4. Semantic guardrails and axiomatic compatibility
5. 5. Transit Boundary Operator
6. 6. Intra-System Boundary Operator
7. 7. Composition and notation behavior
8. 8. Serialization profile and formal grammar
9. 9. JSON mapping and implementation profile
10. 10. Rendering and visualization guidance
11. 11. Worked examples
12. 12. Conformance requirements
13. Appendix A. Symbol table
14. Appendix B. Context vocabulary
15. Appendix C. Intra-system boundary catalog
16. Appendix D. Decision procedure
17. Appendix E. Anti-patterns and misuse cases

1. Abstract

This specification defines two notation extensions for the TLCTC attack-path language: the Transit Boundary Operator and the Intra-System Boundary Operator. The Transit Boundary Operator identifies a third-party intermediary whose infrastructure sits topologically between source and target in the delivery path of a step. The Intra-System Boundary Operator identifies a security-domain crossing that occurs within one system instance during a step.

Both operators are annotations. They add structural precision to attack-path notation, but they do not create new TLCTC clusters, do not redefine existing clusters, and must not alter the classification of any attack step. Classification remains determined solely by the generic vulnerability initially exploited by that step.

This specification is intentionally complete enough for real use: it includes definitions, semantics, normative rules, composition behavior, serialization guidance, a formal extension grammar, a minimal implementation profile, and worked examples.

2. Status, scope, and relationship to TLCTC v2.0

This is a full draft specification for review. It is written as a candidate v2.1 profile that is designed to remain fully compatible with v2.0 notation and v2.0 semantic principles.

This specification covers:

- the syntax and semantics of the Transit Boundary Operator;
- the syntax and semantics of the Intra-System Boundary Operator;

- combined placement rules with existing v2.0 operators;
- canonical Unicode and ASCII transport forms;
- a formal extension grammar sufficient for deterministic parsing of the new operators;
- a minimal JSON mapping and implementation profile;
- conformance requirements and misuse guardrails.

This specification does not:

- define any new TLCTC threat cluster;
- replace the v2.0 causal classification model;
- promote outcomes such as privilege escalation or sandbox escape into threats;
- provide a complete formal grammar for every historical v2.0 construct outside the scope needed for the v2.1 extension profile;
- replace forensic prose, analyst judgment, or evidence statements.

Relationship to v2.0: every valid v2.0 notation string remains valid in v2.1. When v2.1 annotations are stripped, the underlying v2.0 cluster sequence must still be recoverable.

3. Normative language and conformance philosophy

The keywords MUST, MUST NOT, REQUIRED, SHALL, SHALL NOT, SHOULD, SHOULD NOT, RECOMMENDED, MAY, and OPTIONAL are to be interpreted as described in RFC 2119 and RFC 8174 when written in uppercase.

Conformance in this specification is semantic first and syntactic second. A parser or rendering tool is not conformant merely because it accepts symbols. It must preserve the underlying TLCTC meaning: annotations cannot be allowed to mutate cluster identity, to hide distinct causal steps, or to introduce effect-oriented pseudo-categories.

4. Semantic guardrails and axiomatic compatibility

This section is normative. It exists to prevent category drift. The v2.1 operators are valuable only if they remain subordinate to the v2.0 framework rather than silently changing it.

SG-1. Cause first. Every attack step SHALL be classified by the generic vulnerability initially exploited in that step, not by the delivery topology, boundary crossed, internal effect produced, or defender control that failed.

SG-2. Topology is not classification. Transit and intra-system annotations describe structure. They MUST NOT define, imply, or substitute for a TLCTC cluster.

SG-3. Annotations are subordinate. A boundary annotation MAY refine a step, but it MUST NOT appear as an independent attack-path element and MUST NOT replace a cluster identifier.

SG-4. Effects are not threats. Sandbox escape, privilege escalation, process pivot, hypervisor escape, persistence, and data exfiltration are not TLCTC clusters. They are internal transitions, follow-on actions, or outcomes that must remain attached to a #1–#10 cause.

SG-5. Actors are not threats. Transit parties, vendors, cloud providers, carriers, or platform operators are responsibility spheres, not threat categories.

SG-6. Distinct exploit rule. If forensic evidence shows that an internal transition required a separately evidenced exploit of a distinct vulnerability, the notation MUST express a new cluster step.

SG-7. Backward recoverability. Stripping all v2.1 annotations from a valid expression MUST leave a valid underlying cluster sequence or single cluster step.

These guardrails are the main reason the extension remains TLCTC-consistent. Without them, the same notation features that improve fidelity would also create semantic ambiguity.

5. Transit Boundary Operator

5.1 Definition

The Transit Boundary Operator identifies an inter-organizational boundary in which the path of a step traverses infrastructure operated by a third party that is neither the source nor the target. The transit party is topologically in-between.

5.2 Canonical syntax

```
||[context][@Source⇒@Transit→@Target]||
```

5.3 Reading rule

Read the operator as: the step originates in @Source, traverses intermediary infrastructure operated by @Transit, and is delivered to @Target.

5.4 Core semantics

- The operator is about topology, not blame.
- The operator is valid only when the intermediary sits between source and target in the path of delivery or relay.
- The operator does not state that the intermediary is malicious, compromised, or the root cause of the step.
- The operator exists because an intermediary may have distinct security obligations at that boundary, such as validation, abuse detection, relay policy, rate limiting, filtering, or takedown capability.

5.5 Actionable control responsibility

For purposes of this specification, actionable control responsibility means a boundary-specific capacity to prevent, constrain, detect, validate, rate-limit, or terminate abusive use of the intermediary service. It does not determine the TLCTC cluster of the step.

5.6 Normative rules

R-TRANSIT-1. Distinct parties. @Transit **MUST** be distinct from both @Source and @Target.

R-TRANSIT-2. True intermediary topology. The operator **MUST** be used only when the intermediary infrastructure sits between source and target in the step's delivery path.

R-TRANSIT-3. No vendor substitution. The operator **MUST NOT** be used to encode vendor responsibility for code that runs on the target. A browser vendor whose browser is exploited on the victim device is not transit merely because the vendor wrote the code.

R-TRANSIT-4. Control relevance. The operator **SHOULD** be used when the intermediary has meaningful control responsibility at that boundary. It **MAY** be omitted where the intermediary is analytically incidental.

R-TRANSIT-5. Pure conduit fallback. If the intermediary adds no useful control surface, the analyst **MAY** use the binary v2.0 boundary or omit the transit annotation entirely, depending on the scenario.

R-TRANSIT-6. Compromise or coercion is separate. If transit is enabled by coercion, credential misuse, physical compromise, administrative abuse, or supply-chain compromise of the intermediary, that enabling condition **MUST** be modeled as a preceding cluster step. Transit notation alone is insufficient.

R-TRANSIT-7. Cluster independence. Transit annotation **MUST NOT** change cluster classification.

R-TRANSIT-8. Multiple transit parties. If the path traverses multiple distinct intermediaries in sequence, chained transit **MAY** be used, but only when each party has independent analytical relevance.

5.7 Chained transit

```
||[signaling][@Attacker⇒@CarrierB⇒@CarrierA⇒@Target]||
```

Chained transit is intentionally narrow. It is appropriate for paths such as SS7 signaling or multi-stage relays where multiple intermediary operators each have their own control responsibility and the ordering matters.

5.8 Correct and incorrect use

Correct

```
||[messaging][@Attacker⇒@Apple(iMessage)⇒@Target]||
||[auth][@Attacker⇒@IdentityProvider⇒@Target(CloudApp)]||
||[network][@NSO_Operator⇒@MobileCarrier⇒@Target]||
```

Incorrect

```
||[browser][@Attacker⇒@Apple(Safari)⇒@Target]||
```

Reason: Safari runs on the target device. Apple may be the vendor responsible for code quality, but it is not topologically between attacker and target for that exploit step.

5.9 Relationship to #10 Supply Chain Attack

Transit is not a substitute for #10. If the intermediary itself is compromised, poisoned, or abused through a trusted third-party trust link, the enabling compromise step belongs to its own cluster, typically #10, #1, #4, #8, or #9 depending on cause. Transit may still appear after that step if the path then traverses the intermediary as infrastructure.

6. Intra-System Boundary Operator

6.1 Definition

The Intra-System Boundary Operator identifies a security-domain crossing that occurs within one device, host, VM environment, containerized runtime, or other single system instance during an attack step.

6.2 Canonical syntax

```
|[type][@From⇒@To]|
```

6.3 Allowed boundary types in v2.1

- sandbox — an application or container isolation boundary;
- privilege — a privilege-level or authority boundary;
- process — a process isolation boundary;
- hypervisor — a guest-to-hypervisor or hypervisor-to-guest isolation boundary.

Memory-centric notation is intentionally deferred from v2.1. Some hardware-enforced memory domains may warrant future treatment, but probabilistic mitigations and mitigation bypasses do not fit cleanly into the same boundary model and should not be forced into it here.

6.4 Core semantics

- The operator identifies an internal containment boundary crossed during a step.

- The operator does not define the threat category of the step.
- The operator is valuable when forensic or vendor-facing analysis needs to preserve containment-failure structure that would otherwise flatten into a single opaque exploit description.
- At the strategic layer, the base cluster sequence often remains sufficient and the operator may be omitted.

6.5 Normative rules

R-INTRA-1. Single-system scope. The operator **MUST** be used only for boundaries within a single system instance.

R-INTRA-2. Cluster attachment. The operator **MUST** be attached to the cluster step that accomplishes the crossing.

R-INTRA-3. No standalone use. The operator **MUST NOT** appear without an associated cluster step.

R-INTRA-4. No cluster change. The operator **MUST NOT** change cluster classification.

R-INTRA-5. Optional precision. The operator is **OPTIONAL** and is mainly recommended for forensic, exploit-analysis, or vendor-facing use.

R-INTRA-6. Multiple crossings. Multiple intra-system annotations **MAY** follow one step when a compressed representation is justified by the evidence.

R-INTRA-7. Distinct vulnerabilities require distinct steps. If a crossing is achieved by exploiting a separately evidenced distinct vulnerability, a new cluster step **MUST** be added. The annotation alone is insufficient.

R-INTRA-8. Compressed form under uncertainty. If evidence shows multiple internal crossings but does not distinguish separate exploit causes at each crossing, a compressed single-step form **MAY** be used.

R-INTRA-9. Anti-effect rule. Privilege escalation, sandbox escape, process pivot, and hypervisor escape **MUST NOT** be modeled as independent threat categories. Every annotated internal crossing remains subordinate to a #1–#10 cluster step.

6.6 Compressed versus expanded form

Compressed form

```
#3 |[sandbox][@A->@B]| |[privilege][@user->@root]| → #7
```

Expanded form

```
#3 |[sandbox][@A->@B]| → #3 |[privilege][@user->@root]| → #7
```

The expanded form is always safe. The compressed form is acceptable only when evidence does not support a stronger decomposition.

6.7 Relationship to the v2.0 ring model

The privilege boundary type formalizes internal ring-like privilege transitions already implicit in TLCTC's component and protection-boundary thinking. The sandbox and process types extend that logic to modern operating-system isolation primitives that do not map neatly to the classic ring vocabulary.

7. Composition and notation behavior

7.1 Operator inventory

Operator	Meaning	Version
#X → #Y	Sequential attack steps	v2.0
(#X + #Y)	Parallel or combined steps	v2.0

→[Δt=value]→	Temporal gap or velocity annotation	v2.0
+ [DRE: X]	Data risk event annotation	v2.0
[ctx][@A→@B]	Binary inter-organizational boundary	v2.0
[ctx][@A→@T→@B]	Transit boundary	v2.1
[type][@X→@Y]	Intra-system boundary	v2.1

7.2 Canonical placement order

```
#X ||[inter-org boundary]|| |[intra-system boundary]| →[Δt=value]→ #Y + [DRE: X]
```

This order is recommended because it is stable, readable, and easy to parse. Not all elements need to appear in every step. Use only those that add structural clarity.

7.3 Stripping behavior

A v2.1-aware implementation SHOULD be able to remove boundary annotations and leave the cluster sequence intact. For example, stripping the v2.1 annotation from “#3 ||[messaging][@Attacker⇒@Apple(iMessage)→@Target]|| |[sandbox][@A→@B]| → #7” yields “#3 → #7”.

7.4 Responsibility metadata

Vendor responsibility, control ownership, and boundary-enforcement ownership are analytically important, but they are not equivalent to transit topology. This specification therefore treats responsibility metadata as separate from notation semantics, even when the same named party appears in both contexts.

8. Serialization profile and formal grammar

8.1 Canonical Unicode profile

Canonical output SHOULD use Unicode forms for the arrow and transit arrow. Input parsers MUST accept both Unicode and the ASCII transport profile defined below.

Concept	Unicode	ASCII transport
Sequential arrow	→	->
Transit arrow	⇒	=>
Delta-t	Δt	dt
Inter-org delimiter		
Intra-system delimiter		

8.2 Parenthetical detail rules

A party identifier may optionally carry parenthetical detail, for example @Apple(iMessage) or @Target(Safari). Parenthetical content is a precision aid, not a second namespace.

- Permitted: letters, digits, spaces, periods, hyphens, underscores, and the @ sign where needed inside parenthetical email content.
- Prohibited: parentheses, square brackets, pipes, arrows, commas, and semicolons.
- Dot notation may be used for nested services such as @Microsoft(AzureAD.OAuth) or @Apple(iCloud.Photostream).

8.3 Delimiter disambiguation

Two consecutive pipes with no intervening content denote the inter-organizational operator. A single pipe followed by a bracket denotes the intra-system operator. Operators must not be nested inside each other.

8.4 Formal extension grammar

The following grammar is normative for the v2.1 extension profile. It is intentionally scoped to the new operators and their composition with the core step skeleton rather than every historical formatting variation that may exist in prose.

```

attack-path      = step *( ws? seq ws? step ) [ ws? dre ]
step             = cluster [ ws boundary ] [ ws intra-list ]
seq             = ">" / ">" / velocity
velocity        = (">" / ">") "[" ("Δt" / "dt") "=" value "]" (">" / ">")
cluster         = "# 1*2DIGIT
dre             = "+" ws? "[DRE:" ws? 1*ALPHA "]"
boundary        = inter-boundary
inter-boundary  = "||" "[" context "]" "[" transit-chain "]" "||"
transit-chain   = party arrow party *( transit-arrow party )
arrow           = ">" / ">"
transit-arrow   = "=>" / "=>"
party           = "@" identifier [ "(" detail ")" ]
intra-list      = intra *( ws intra )
intra           = "|" "[" boundary-type "]" "[" party arrow party "]" "|"
context        = identifier
boundary-type   = "sandbox" / "privilege" / "process" / "hypervisor"
identifier      = 1*( ALPHA / DIGIT / "_" / "-" / "." )
detail         = 1*( ALPHA / DIGIT / "_" / "-" / "." / SP / "@" )
value          = 1*( ALPHA / DIGIT / "." / "≈" / ":" )
ws             = 1*( SP / HTAB )

```

Parsing note: the grammar is deliberately permissive in the value field so that human-readable velocity tokens such as 5m, 3–5s, or ≈0s remain representable. Implementations that require stronger machine validation may layer a stricter local schema on top of this profile.

8.5 Canonicalization rules

CAN-1. Canonical output SHOULD normalize arrows to Unicode.

CAN-2. Canonical output SHOULD remove redundant whitespace while preserving a single space between major step components.

CAN-3. Canonical output SHOULD place inter-organizational boundary annotations before intra-system annotations on the same step.

CAN-4. Canonical output SHOULD preserve party detail as supplied, except for removal of prohibited characters.

9. JSON mapping and implementation profile

9.1 Minimal object model

```

{
  "step_id": "S2",
  "cluster": "#3",
  "description": "iMessage parser vulnerability exploited",
  "inter_org_boundary": {

```



```

    "context": "messaging",
    "source": "@Attacker",
    "transit": "@Apple(iMessage)",
    "target": "@Target"
  },
  "intra_system_boundaries": [
    { "type": "sandbox", "from": "@iMessage", "to": "@BlastDoor" },
    { "type": "privilege", "from": "@user", "to": "@root" }
  ],
  "vendor_responsibility": {
    "vendor": "Apple",
    "components": ["iMessage parser", "iOS privilege separation"],
    "note": "Metadata only; not transit topology"
  }
}

```

9.2 JSON rules

JSON-1. The `inter_org_boundary` object is OPTIONAL.

JSON-2. Within `inter_org_boundary`, the `transit` field is OPTIONAL. If absent, the boundary is binary v2.0-compatible.

JSON-3. The `intra_system_boundaries` array is OPTIONAL and may contain multiple entries for compressed form.

JSON-4. `vendor_responsibility` is OPTIONAL metadata and has no effect on parser classification.

JSON-5. A compliant implementation MUST preserve the cluster independently of all annotation metadata.

9.3 Implementation behavior

- A parser SHOULD accept both Unicode and ASCII transport forms.
- A renderer SHOULD be able to display both inter-organizational and intra-system boundaries distinctly.
- An exporter SHOULD be able to strip annotations and emit the base cluster sequence.
- A validator SHOULD warn when `transit` is used for a vendor-on-target case or when an intra-system boundary appears without a cluster.

10. Rendering and visualization guidance

This section is informative but recommended. The notation is human-first. Visual clarity therefore matters.

- Inter-organizational boundaries are best rendered as vertical swim-lane boundaries or labeled boundary markers between responsibility spheres.
- Transit parties should appear between source and target lanes, not merged into either endpoint.
- Intra-system boundaries are best rendered as horizontal sub-lanes or compartment lines inside the target's lane.
- The transit arrow should be visually distinguishable from the ordinary sequential arrow.
- Compressed and expanded intra-system forms should remain visually recognizable as different levels of evidentiary certainty, not just different art styles.

11. Worked examples

11.1 Pegasus network injection

```
#5 |[network][@NSO_Operator⇒@MobileCarrier⇒@Target]|| →[Δt=0s]
#3 |[browser][@NSO(free247downloads.com)⇒@Target(Safari)]|[privilege][@user⇒@root]|
→[Δt=0s]
#7 + [DRE: C]
```

Interpretation: the mobile carrier is transit for the network redirection step. Safari is not transit; it is the target-side client component exploited in the #3 step. The privilege transition is an internal crossing annotated on the exploit step. Malware execution remains #7.

11.2 iMessage zero-click, expanded form

```
#3 |[messaging][@Attacker⇒@Apple(iMessage)⇒@Target]||
|[sandbox][@iMessage⇒@BlastDoor]| →
#3 |[process][@BlastDoor⇒@WebKit.WebContent]| →
#3 |[process][@WebKit.WebContent⇒@coretelephony]| |[privilege][@user⇒@root]| →
#7 + [DRE: C]
```

Interpretation: each new #3 step is justified only when evidence supports a distinct vulnerability-driven internal transition. The internal crossings remain effects and containment failures attached to #3, not new threat categories.

11.3 OAuth phishing path

```
#9 |[email][@Attacker⇒@Microsoft(M365)⇒@Target]|| →[Δt=hours]
#4 |[auth][@Attacker⇒@AzureAD.OAuth⇒@Target(CloudResources)]|| →[Δt=0s]
#1 + [DRE: C]
```

Interpretation: the email provider and identity provider are topological intermediaries. The use of stolen or coerced authorization remains #4. Subsequent abuse of legitimate cloud functions remains #1.

11.4 SS7 interception and account use

```
#5 |[signaling][@Attacker⇒@CarrierB(SS7)⇒@CarrierA⇒@Target]|| →[Δt=0s]
#4 →[Δt=0s] #1 + [DRE: C]
```

Interpretation: chained transit is justified because two carriers each hold distinct control responsibility in the signaling path. Credential use remains #4, and follow-on misuse of legitimate application functions remains #1.

11.5 VM escape in shared cloud

```
#2 |[hypervisor][@GuestVM⇒@Hypervisor]| →[Δt=0s]
#1 |[hypervisor][@Hypervisor⇒@VictimVM]|
|[cloud][@Attacker(Tenant)⇒@CloudProvider⇒@Victim(Tenant)]|| + [DRE: C]
```

Interpretation: the hypervisor flaw is a #2 step. The later abuse of legitimate hypervisor or management functions is #1. The cloud provider is transit for the cross-tenant boundary, but not the threat category.

11.6 Local script to SYSTEM via misconfiguration

```
#7 → #1 |[privilege][@user⇒@SYSTEM]|
```

Interpretation: the notation makes clear that the system privilege state changes, but the cause remains abuse of a legitimate local function or configuration weakness rather than an invented “privilege escalation cluster”.

11.8 Pegasus Photostream delivery

```
#1 |[cloud][@Attacker(bogaardlisa803@gmail.com)⇒@Apple(iCloud.Photostream)→
@Target(Photos)]| →[Δt≈0s]
#3 |[privilege][@user→@root]| →[Δt≈0s]
#7 + [DRE: C]
```

Interpretation: the attacker abuses the designed auto-delivery scope of Apple's iCloud Photostream service to push crafted media to the target device. This is #1 because Photostream functions as designed — no code flaw is exploited in the delivery mechanism itself. Apple's iCloud service is transit because its relay infrastructure sits between the attacker's iCloud account and the target's device. The #3 step is the client-side parsing flaw in the Photos app that triggers the unintended data→code transition. The privilege crossing is annotated on the #3 step. Note that #1 at a transit boundary is structurally valid: the transit operator marks topology, the cluster marks cause, and the two are independent per SG-2.

12. Conformance requirements

An implementation conforms to this specification if it satisfies all of the following:

1. accepts all valid v2.0 notation without semantic change;
2. accepts both Unicode and ASCII transport forms for the v2.1 operators;
3. parses the transit operator only as a topological intermediary annotation;
4. parses the intra-system operator only as a cluster-attached internal-boundary annotation;
5. preserves cluster classification independently from all annotations;
6. supports compressed and expanded intra-system expression without conflating them;
7. can strip v2.1 annotations and recover the underlying cluster sequence;
8. rejects or warns on standalone intra-system boundaries or transit misuse for vendor-on-target cases.

A human analyst may still produce semantically poor notation even if the parser accepts it. Implementations are encouraged to provide linting rules in addition to pure syntax validation.

Appendix A. Symbol table

Symbol	Meaning
→	Sequential step ordering or directional relationship
⇒	Transit relationship to a topological intermediary
	Inter-organizational boundary delimiter
	Intra-system boundary delimiter
Δt	Time gap or velocity annotation
+	Parallel combination or DRE attachment
@	Responsibility sphere prefix
#	TLCTC cluster prefix

Appendix B. Context vocabulary

The context vocabulary is extensible. The list below defines a practical baseline profile.

Context	Typical meaning	Typical transit party
messaging	Message relay or delivery platform	Platform operator
email	Email transport and delivery	Email provider
cdn	Content delivery infrastructure	CDN provider
cloud	Cloud control or service infrastructure	Cloud provider
network	Network-level relay or path control	ISP, carrier, VPN provider
signaling	Telephony signaling path	Carrier(s)
auth	Identity federation or authentication relay	Identity provider
media	Media streaming or content relay	Media platform
browser	Client browser content delivery	Usually binary; transit only if separate relay exists
exploit	Direct exploit delivery path	Usually binary; transit only when relay infrastructure matters
dev	Development or build path	Vendor or build service
update	Software update delivery path	Update service provider
physical	Physical access channel	Facility or hardware operator
human	Human communication channel	Organization or operator
legal	Compulsion or legal order channel	State or regulator
admin	Administrative access channel	System operator

Appendix C. Intra-system boundary catalog

Type	Definition	Examples
------	------------	----------

sandbox	Application or container isolation boundary	@iMessage, @BlastDoor, @Chrome, @Docker
privilege	Privilege or authority boundary	@user, @root, @admin, @SYSTEM, @Ring0
process	Process isolation boundary	@WebKit.WebContent, @coretelephony, @renderer
hypervisor	Guest, host, or hypervisor isolation boundary	@GuestVM, @Hypervisor, @Host

Deferred from v2.1: memory-oriented notation. Hardware-enforced memory compartments and mitigation bypasses may warrant future treatment, but they are intentionally excluded here to avoid mixing isolation domains with mitigation states.

Appendix D. Decision procedure

Use transit when all of the following are true:

1. the step crosses an inter-organizational boundary;
2. three or more responsibility spheres are structurally involved;
3. one party is topologically between source and target;
4. the intermediary has meaningful control relevance.

Do not use transit when the named party is merely the vendor of code running on the target.

Use the intra-system operator when all of the following are true:

1. the step crosses a meaningful internal containment boundary within one system;
2. naming that boundary adds analytical value;
3. the annotation remains attached to a cluster step.

If a separately evidenced vulnerability is required for the next internal crossing, create a new cluster step.

Appendix E. Anti-patterns and misuse cases

- A1.** Do not write a path where `[[privilege][@user->@root]]` appears without a cluster. That turns an effect into a pseudo-threat.
- A2.** Do not use transit to encode vendor blame for a target-side component. Vendor responsibility belongs in prose or metadata.
- A3.** Do not compress multiple clearly distinct exploit causes into one cluster step simply because the chain remained inside one device.
- A4.** Do not let a platform operator's presence trick you into #10 unless a trust-link compromise is actually the cause.
- A5.** Do not let modern exploit vocabulary such as sandbox escape or process pivot replace TLCTC cluster labels.

Final normative statement: the v2.1 extension is valid only if the threat cluster remains determined by the generic vulnerability exploited, never by the path topology, the boundary crossed, the vendor involved, or the downstream consequence produced.